Compte-Rendu de TP : Installation, Configuration et Sécurisation MQTT sur Ubuntu/Kali

Date : 31 Mars 2026
Auteur : Étudiant IPSSI - MSC Cybersécurité
Sujet : Sécurisation des communications IoT via le protocole MQTT

---

1. Introduction et Objectifs

Ce TP a pour objectif la mise en place d'un environnement IoT sécurisé basé sur le broker **Mosquitto**. Les principaux axes de travail sont :

- L'installation et la configuration initiale d'un broker MQTT.
- La mise en œuvre de l'authentification par mot de passe.
- La sécurisation des flux via le chiffrement **TLS**.
- Le contrôle d'accès granulaire via les **ACL** (Access Control Lists).
- Le déploiement conteneurisé via **Docker Compose**.

---

2. Environnement de Travail

- **Système d'exploitation :** Kali Linux (VM)
- **Broker MQTT :** Mosquitto v2.0.11
- **Outils :** `mosquitto_pub`, `mosquitto_sub`, `openssl`, `docker-compose`.

---

3. Étapes de Réalisation

3.1 Préparation du Système

La première étape consiste à mettre à jour le système et installer les dépendances nécessaires.

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update && sudo apt upgrade -y
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y mosquitto mosquitto-clients docker.io docker-compose
openssl net-tools
```

3.2 Installation et Test Initial

Après l'installation, nous vérifions que le service est actif et qu'il écoute sur le port par défaut (1883).

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable --now mosquitto
┌──(kali㉿kali)-[~]
└─$ ss -tulnp | grep 1883
tcp   LISTEN 0      100        127.0.0.1:1883       0.0.0.0:*
users:(("mosquitto",pid=1234,fd=5))
```

Test de communication (Anonyme par défaut) : Dans un premier terminal, nous souscrivons à un topic :

```
┌──(kali㉿kali)-[~]
└─$ mosquitto_sub -h localhost -t "test/topic" -v
```

Dans un second terminal, nous publions un message :

```
┌──(kali㉿kali)-[~]
└─$ mosquitto_pub -h localhost -t "test/topic" -m "Hello MQTT"
```

Résultat : Le message est bien reçu par le souscripteur.

---

4. Sécurisation du Broker

4.1 Authentification par Mot de Passe

Pour interdire l'accès anonyme, nous créons un fichier de mots de passe et configurons Mosquitto.

```
┌──(kali㉿kali)-[~]
└─$ sudo mosquitto_passwd -c /etc/mosquitto/passwd user1
Password: password123
Reenter password: password123
```

Modification de `/etc/mosquitto/mosquitto.conf` :

```
allow_anonymous false
password_file /etc/mosquitto/passwd
```

Validation :

```
# Tentative anonyme (Échec attendu)
┌──(kali㉿kali)-[~]
└─$ mosquitto_pub -h localhost -t "test" -m "fail"
Connection error: Connection Refused: not authorised.

# Tentative avec user1 (Succès)
┌──(kali㉿kali)-[~]
└─$ mosquitto_pub -h localhost -u user1 -P password123 -t "test" -m
"success"
```

4.2 Chiffrement TLS

Le protocole MQTT transmet les données en clair par défaut. Pour contrer les attaques de type **Man-In-The-Middle (MITM)**, nous activons le TLS sur le port 8883.

Génération des certificats :

```
┌──(kali㉿kali)-[~]
└─$ sudo mkdir -p /etc/mosquitto/certs
┌──(kali㉿kali)-[~]
└─$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/mosquitto/certs/server.key \
    -out /etc/mosquitto/certs/server.crt \
    -subj "/CN=localhost"
```

Configuration du listener TLS :

```
listener 8883
cafile /etc/mosquitto/certs/server.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key
```

Test TLS :

```
┌──(kali㉿kali)-[~]
└─$ mosquitto_pub -h localhost -p 8883 --cafile
/etc/mosquitto/certs/server.crt \
    -u user1 -P password123 -t "test/tls" -m "Message Sécurisé"
```

4.3 Contrôle d'Accès (ACL)

Les ACL permettent de définir précisément qui peut lire ou écrire sur quel topic.

Fichier `/etc/mosquitto/aclfile` :

```
user user1
topic readwrite test/#
topic read maison/temperature
```

Test des ACL :

```
# Publication autorisée sur test/#
┌──(kali㉿kali)-[~]
└─$ mosquitto_pub -u user1 -P password123 -t "test/data" -m "OK"

# Publication INTERDITE sur maison/temperature (Lecture seule)
┌──(kali㉿kali)-[~]
└─$ mosquitto_pub -u user1 -P password123 -t "maison/temperature" -m "25"
# Le message est ignoré par le broker (ou déconnecté selon la version)
```

---

5. Déploiement Docker Compose

Pour industrialiser le déploiement, nous utilisons un fichier `docker-compose.yml`.

```yaml
version: '3.8'
services:
  mqtt-broker:
    image: eclipse-mosquitto:2.0
    ports:
      - "1883:1883"
      - "8883:8883"
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - ./passwd:/etc/mosquitto/passwd:ro
      - ./certs:/etc/mosquitto/certs:ro
```

Lancement :

```
┌──(kali㉿kali)-[~/tp_mqtt]
└─$ docker-compose up -d
┌──(kali㉿kali)-[~/tp_mqtt]
└─$ docker-compose logs -f
```

---

6. Analyse de Sécurité et Réponses aux Questions

6.1 Tableau des Vulnérabilités et Mitigations

Vulnérabilité	Risque	Mitigation Appliquée
Accès Anonyme	Prise de contrôle, injection de fausses données.	`allow_anonymous false` + Fichier de mots de passe.
Flux en clair	Interception des identifiants et données (MITM).	Mise en œuvre du protocole TLS sur le port 8883.
Topics permissifs	Fuite d'informations, modification non autorisée.	Configuration granulaire via un fichier ACL.

6.2 Réponses aux Questions du TP

1. Quels sont les risques si MQTT n'est pas sécurisé ? Sans sécurité, n'importe quel attaquant sur le réseau peut :

- Intercepter les données sensibles des capteurs.
- Envoyer des commandes malveillantes aux actionneurs (ex: ouvrir une serrure connectée).
- Réaliser un déni de service (DoS) en saturant le broker.

2. Quelle est la différence entre TLS et mTLS ?

- **TLS (Transport Layer Security) :** Seul le serveur prouve son identité au client via un certificat. La communication est chiffrée.
- **mTLS (Mutual TLS) :** Le serveur ET le client doivent présenter un certificat valide. Cela renforce l'authentification en s'assurant que seuls les appareils autorisés

(possédant la clé privée correspondante) peuvent se connecter.

3. Pourquoi les ACL sont-elles importantes ? Les ACL appliquent le principe du **moindre privilège**. Même si un compte utilisateur est compromis, l'attaquant sera limité aux topics autorisés pour cet utilisateur, empêchant ainsi une compromission totale du système IoT.

---

7. Conclusion

Ce TP a permis de transformer un service MQTT vulnérable par défaut en une infrastructure robuste. La combinaison de l'authentification forte, du chiffrement des flux et du contrôle d'accès granulaire constitue la base indispensable de toute architecture IoT sécurisée.